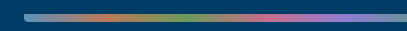


CHAPTER 12 OF 13 • PART SIX

Designing a Parking App

A driver scans a sticker on the bay and pays. One table, three rungs, and the bill is not the one you expected.



Thirteen chapters, gathered into six groups.

01 A Tuesday — one engineer, wake to sleep

02 The words — naming what you just watched

03 Protagonist and antagonist — where a design starts

04 Solution types — which answer is wanted

05 Six kits — data, compute, network, scale, reliability, security

→ **Two designs, then the map back — Instagram, a parking app, and what you keep**

WHERE THIS SITS

Chapter eleven designed at the top. This one climbs the ladder.

Same method, smaller product, and nothing is skipped on the way up: a one-table MVP, then a scaled rung, then an optimized one. The payment path is worked to the point you could implement it — deduplicated taps, a unique index over the bay, a webhook that writes only while the row still waits, and a sweep that asks the provider rather than guessing what silence means.

01

PART SIX

Now run the whole method again, on something you could ship this month.

A lot owner wants drivers to scan a sticker on the bay and pay to park.

No app to download. No account to create. A driver walks up, points a phone at a sticker, pays, and walks away.

Instagram was one design at one rung, and it was already enormous when we met it. This one starts at nothing and climbs, which is what your first year actually looks like.

PARKING • THE CAST

The driver has one hand free and about thirty seconds of patience.

THE PROTAGONIST

A driver who has already parked, standing in the rain with a phone in one hand. They want to pay and walk away. They will not install anything and they will not make an account.



THE ANTAGONIST

The garage is underground and the signal is poor. A driver whose page hesitates taps Pay again, so the same park can be charged twice.

PARKING • THE WORKSHEET, FILLED

You fill in eight of the nine before a single box goes on the board.

Protagonist	A driver at the bay. One hand, thirty seconds, no app.
Antagonist	Poor signal underground, and a second tap on Pay.
Purpose	The car is paid for before the driver walks away.
Boundary	In: bays, sessions, payments, enforcement. Out: the card network.
Environment	Rain, cold hands, a low battery, a sticker somebody peeled.
Constraints	physical: no signal economic: card fees human: thirty seconds legal: refunds on request
Invariants	One park charges once • an expiry never extends by accident
Bottleneck	Suspected: none yet. Confirmed once wardens start asking.
Solution type	MVP. Nobody knows yet whether drivers scan the sticker.

Rung one is one lot, a printed sticker per bay, and the provider's card form.

What you build

A sticker per bay whose link carries the lot and bay. The card goes to the provider's form, not your server.

What it buys

The only answer you need this month: do drivers scan the sticker, and do they finish paying.

What it charges

A warden walks the lot typing bay numbers into a phone. That holds at one lot and gives out at ten.

One table holds all of it: no partitioning, no cache, no queue.

sessions

id	uuid. also what you send in the provider's metadata
idem_key	unique. one attempt, one key, however many taps
lot_id, bay	unique while status is waiting. two scans cannot both be live
status	waiting, then paid, declined, refunded, or written off
created_at	set on insert. how you tell a stale wait from a fresh one
minutes	what the driver bought. on insert, so expires_at can be computed
amount_cents	what that costs. on insert, so a retry sends the same request
started_at	when the payment cleared, read off the charge, not your clock
expires_at	started_at plus minutes
payment_ref	the provider's id for the charge

Two hundred lots of forty bays turning over four times a day is 32,000 rows — one machine, for years.

Traffic breaks nothing here. Two other things break anyway.

Thirty-two thousand rows a day is a rounding error, so scale is not your problem and will not be for a long time. Say that out loud, because it stops a team building for a load that never arrives.

Nothing that broke Maya's Tuesday was load either — a registry, a build and a sleeping reviewer, and not one of them was traffic.

The first break is a double charge. The page hesitates on a weak signal, the driver taps Pay again, and one park costs them twice. That reaches a human the same day.

The second is the signal itself. The card form sits on the far side of a network that keeps disappearing.

PARKING • RUNG ONE, CHARGING ONCE

Two taps on Pay have to produce one charge, and a retry must not add another.

The second tap is a different request that means the same thing. So the page mints one key for this attempt and keeps it across reloads, and the unique index decides — not your code.

THE TELL

The page hesitates on a weak signal, and the driver taps Pay a second time.

INSERT FIRST, THEN CHARGE

The key sits in a unique column, so the second tap conflicts instead of paying again. Read the row: paid hands back the receipt, waiting waits for the first answer.

ONE KEY, ONE STORED ANSWER

The provider replays the first result, so a crash costs nothing. It replays a decline too, so the page mints a fresh key before the driver can try again.

The phone can drop off after the card is charged, and it often does.

The card network answers your payment provider, not the driver's phone. So the provider calls you back on a webhook, and that call is what marks the session paid when it arrives.

The row flips to paid and the warden sees a paid bay, whether or not the phone ever came back.

A decline is an answer, not a gap: the webhook writes it down and the bay frees at once. Only a row that got no answer at all — a closed tab, a webhook that never came — is one you sweep. Sweep every few minutes, because a waiting row holds the bay against the next driver.

PARKING • RUNG ONE, WHAT THE SWEEP
DOES

A sweep is a second writer, and it writes on a guess about what silence means.

Ask the provider what happened and write the answer down — the webhook is the fast path to paid, not the only one. But you are writing rows another process writes too, and you are doing it without being sure the driver has gone.

THE TELL

A row has sat waiting fifteen minutes — far longer than a payment takes, even on a bad signal — and nobody has told the driver anything.

WRITE ONLY IF IT IS STILL WAITING

The same guard the webhook needs, for the same reason: two writers on one row, and the later must not bury what the earlier learned.

TO FREE A BAY, CANCEL FIRST

A timeout is not the same answer as nothing was taken. An open payment can still complete, so cancel it at the provider before writing the row off — or you free the bay and take the money afterwards.

PARKING • RUNG ONE, THE DRIVER'S
OWN RETRY

**The warden can see
the bay is paid. The
driver cannot, so the
driver rescans.**

A spinner tells the driver that nothing
landed. They close the tab and scan the
sticker again — a fresh page, a fresh key,
which is exactly why the key does not stop
the second charge.

THE TELL

A driver taps Pay, watches nothing land, and starts over from the sticker.

THE KEY CANNOT STOP THIS

It was minted for one attempt, and a
rescan is a new one. There is nothing
for it to conflict with.

THE BAY CAN

Read the live session first, and hand
back its receipt if it is paid. A unique
index on lot and bay stops a second
scan inserting while the first is still
waiting.

PARKING • RUNG ONE, THE CALLBACK YOU DID NOT WRITE

Anyone on the internet can call that webhook, and your provider will call it twice.

Check the signature your provider sends before you believe a single field in it. Skip that and you have built a free parking machine: anyone who can post to the endpoint can mark any bay paid.

Then expect the same call more than once, because a provider retries until you answer. Find the row by the key the charge carried, and write only if it is still waiting. An update with no condition lands on a row you already refunded and quietly marks it paid again.

PARKING • RUNG ONE, THE KEY EXPIRES

A key only works for a day, so a stuck row is a deadline.

Every provider puts a lifetime on the key — Stripe's is twenty-four hours. Inside that window a retry is free; past it the same key buys a second charge, so look yours up and sweep well inside it.

THE TELL

A row still says waiting the morning after the park, and nobody has told the driver anything.

INSIDE THE WINDOW, THE KEY STILL HOLDS

The provider replays its first answer to that key and those parameters, so a retry inside the day cannot charge twice.

PAST IT, RECONCILE

Never re-send. Look for the charge by the id in the provider's metadata. Refund what you cannot deliver; write the row off if there is nothing there.

YOUR TURN

Wardens arrive. Say what they ask, how often, and what answers it.

Two hundred lots have gone live. A warden walks a lot of forty bays and needs to know which cars are paid for right now.

Write down the one question they ask the system, roughly how often it is asked, and the one index that answers it. Then turn the page.

ONE ANSWER

The warden asks one small question, and it stays small.

- 01

The question

Is bay 12 in lot 40 paid at this moment. One row, never a list.
-
- 02

How often

Two bays a minute per warden, two hundred lots: about 400 a minute. Under ten a second.
-
- 03

What answers it

An index on lot, bay and expiry, over the paid rows only. The question is a point read and it stays one.

Two hundred lots, and the manual parts give out before the machine does.

What actually changed

Not the traffic. The manual work. A warden typing bay numbers gave out at ten lots; Maya's team, at one reviewer.

Give the warden a list, not a keyboard

The point read becomes one range scan per walk: every live session in this lot. Their phone already knows the bays.

Move slow work off the path a driver waits on

Owner reports go to a read replica. Receipts and nightly payouts go behind a bounded queue.

PARKING • RUNG THREE, OPTIMIZED

13%

What the card fee takes from a three-dollar park, at thirty cents plus 2.9 percent.

Servers cost pennies

Thirty-two thousand rows a day runs on the smallest machine sold. Tuning it saves nothing worth having.

The fee is the bill

Thirty-nine cents of every three dollars, and most of it is a flat charge per transaction.

So batch the regulars

Hold one driver's card and capture their four parks as one twelve-dollar charge: 65 cents, not a dollar fifty-five. Four different drivers are still four charges.

And a capture is not free

The fee rides the capture, never the authorization, so charging a car that has already left is a decline to chase, and it needs the account this product does not have yet.

PARKING • WHAT WE REFUSED

A junior would build these first, and only one of them has arrived yet.

REFUSED	WHY	WHAT WOULD EARN IT
A mobile app	A driver in the rain will not install one	Regulars who park daily, once they exist
Accounts and login	A screen between the sticker and the money	Rung three's settlement earned it
A live map of free bays	Needs a sensor in every bay	Somebody willing to pay for the sensors
Knowing which car is in the bay	The session is keyed to the bay, so the next driver parks on what the last one paid for	A plate or a sensor, once the free parks cost more than the hardware

PARKING · THE LADDER, CLIMBED

One product climbed three rungs, and we skipped nothing on the way up.

- | | | |
|----|-----------|--|
| 01 | MVP | One lot, one table, a card form. Bought the answer to "will anyone scan this". |
| 02 | Scaled | Two hundred lots. A replica for reports, a queue for receipts. Nothing sharded. |
| 03 | Optimized | Batched charges, because the profile said the fee was the bill and the servers never were. |

Five moves carried this design, and every one of them came out of a kit.

Relational, because nothing here outgrows one machine and the questions keep changing — pass one ended there. Indexes doing three jobs: the key that stops a second tap, the bay that stops two scans racing, and the one a warden's question needs. Idempotency behind the first, and behind a webhook your provider will send again. A read replica, to keep reports off the path a driver waits on. A bounded queue, for the work nobody is waiting for.

The security kit arrived as practice, not a card: the provider's form keeps card numbers off your servers, and a signed webhook keeps a stranger from marking bays paid. Not one of those is a product name, and not one of them was a guess.

CHAPTER 12 OF 13 • HOW TO THINK ABOUT SYSTEMS

Two designs are done.

Chapter thirteen reads them back: it maps the feed design onto the kits, including the entry we refused, runs the removal test on it, and hands you the worksheet.



Glossary

M

TERM	DEFINITION
MVP	The smallest build that puts a real answer in front of a real user.